

Increment ++ and Decrement -- Operator in C++

In C++, we use the **increment** ++ and **decrement** -- operators to **increase** and **decrease** the value of a variable **by one** respectively.

```
int x = 10; // 10 x
              4B
x = x + 1;
or
x++; ← post incr. op
or
++x; ← pre incr. op
cout << x; // 11
```

```
x = 10; // 10 x
cout << x++; // 10
cout << x; // 11
```

```
int a = 50; // 50 a
int b = a++; // 50 b
cout << a; // 51
cout << b; // 50
```

```
int z = 90;
z = z - 1;
or
z--; // post decr. op
or
--z; // pre decr. op
cout << z; // 89
```

```
int y = 20; // 20 y
cout << ++y; // 21
cout << y; // 21
```

```
int c = 100; // 100 c
int d = ++c; // 101 d
cout << c; // 101
cout << d; // 101
```

```

a  b
0  11 ✓
0  10

int main() {
    int a = 0;
    int b = 10;
    if(a > 0 and a++) {
        a++;
    } else {
        b++;
    }
    cout << a << " " << b << endl;
    return 0;
}
```

Handwritten notes: a: 0, b: 10. 0 10. 0 > 0 and a++ is false. a++ is not eval. b is true.

```

a  b
1  10 ✓
2  10

int main() {
    int a = 0;
    int b = 10;
    if(b > 0 or a++) {
        a++;
    } else {
        b++;
    }
    cout << a << " " << b << endl;
    return 0;
}
```

Handwritten notes: a: 0, b: 10. 0 10. 10 > 0 or a++ is true. a++ is not eval. b is true.

a b
2 11
0 12
1 11
1 12 ✓

```
int main() {
    int a = 0;
    int b = 11;

    if(b < 0 or a++) {
        a++;
    } else {
        b++;
    }

    cout << a << " " << b << endl;
    return 0;
}
```

1 ☒ 0 ☒ 12
a b
11 < 0 or a++
false
false

```
int main() {
    int a = 0;
    int b = 11;

    if(b < 0 or ++a) {
        a++;
    } else {
        b++;
    }

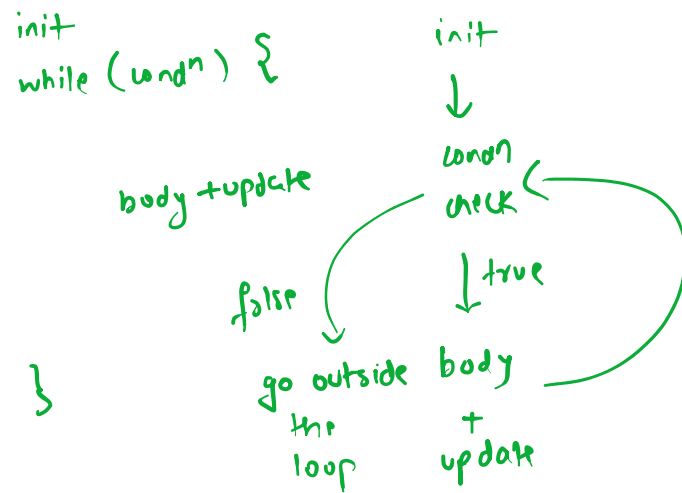
    cout << a << " " << b << endl;
    return 0;
}
```

2 ☒ 0 ☒ 11
a b
11 < 0 or ++a
false (true)
true

a b ✓
2 11
1 11
0 12

➤ The for Loop

```
int main() {
    // ...
    for(init; condition; update) {
        // body of the loop
    }
    // ...
    return 0;
}
```



```
int main() {
    int n;
    cin >> n;
    for(int i=0; i<n; i++) {
        cout << i << " ";
    }
    cout << endl;
    return 0;
}
```

```
int main() {
    int n;
    cin >> n;
    int i=0;
    while(i < n) {
        cout << i << " ";
        i++;
    }
    cout << endl;
    return 0;
}
```

n=5
i=0,1,2,3,4,5
0<5? ✓
1<5? ✓
2<5? ✓
3<5? ✓
4<5? ✓
5<5? x
op: 0,1,2,3,4

while vs for Loop

- Use a **while** loop if the loop condition or the update rule is **complex**.
- Use a **for** loop to iterate over a **sequence** of items.

➤ [HW][R]The **do-while** Loop

```
int main() {  
    // ...  
    // initialization  
    do {  
        // body of the loop  
    } while (condition);  
    // ...  
    return 0;  
}
```

```
int main() {  
    int n;  
    cin >> n;  
    int i = 0;  
    do {  
        cout << i << " ";  
        i++;  
    } while(i < n);  
    cout << endl;  
    return 0;  
}
```

Unlike a **while** or a **for** loop, the body of the **do-while** loop will run **at-least once** irrespective of the loop condition.

```
int main() {  
    int i = 5;  
    do {  
        cout << i << " ";  
        i++;  
    } while(i < 5);  
    cout << endl;  
    return 0;  
}
```

We can **simulate** a do-while loop using a while loop.

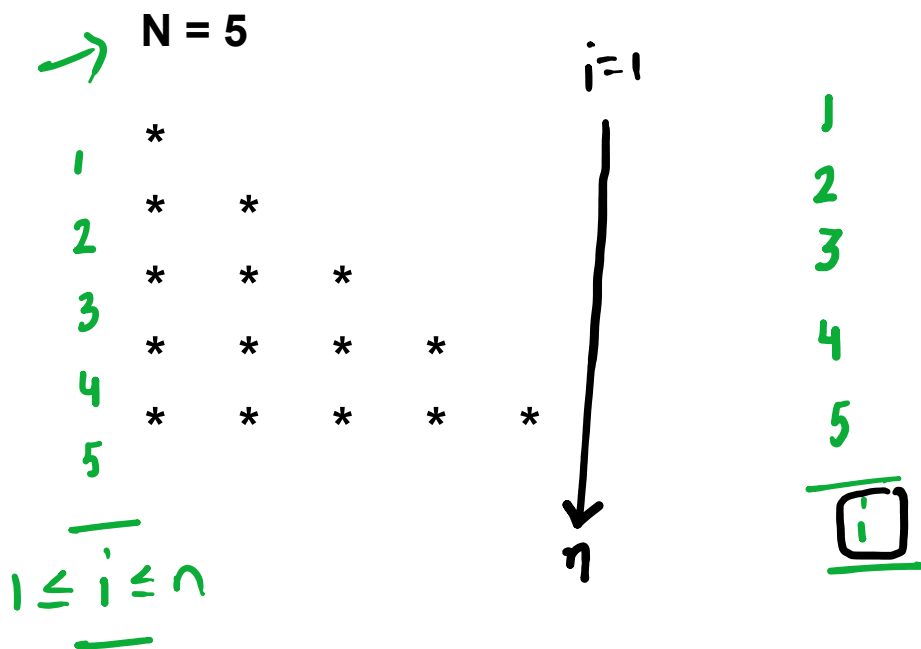
```
int main() {  
    // ...  
    // initialization  
    do {  
        // body of the loop  
    } while (condition);  
    // ...  
    return 0;  
}
```

```
int main() {  
    // ...  
    // initialization  
    while(true) {  
        // body of the loop  
        if(condition) {  
            break;  
        }  
    }  
    // ...  
    return 0;  
}
```

Pattern - 1

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example



Pattern - 2

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

1	1				
2	1	2			
3	1	2	3		
4	1	2	3	4	
5	1	2	3	4	5

1 ≤ i ≤ n

i = 1

↓

n

1
2
3
4
5

i

→

for the ith row
print i nos.
in ring order
st. with 1

Pattern - 3

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 4

```
1
2 3
4 5 6
7 8 9 10
```


Pattern - 4

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

→ 1
→ 2
→ 3
→ 4
→ 5

```

1
0 1
0 1 0 1
1 0 1 0 1

```

i

↓
n

1
2
3
4
5
i

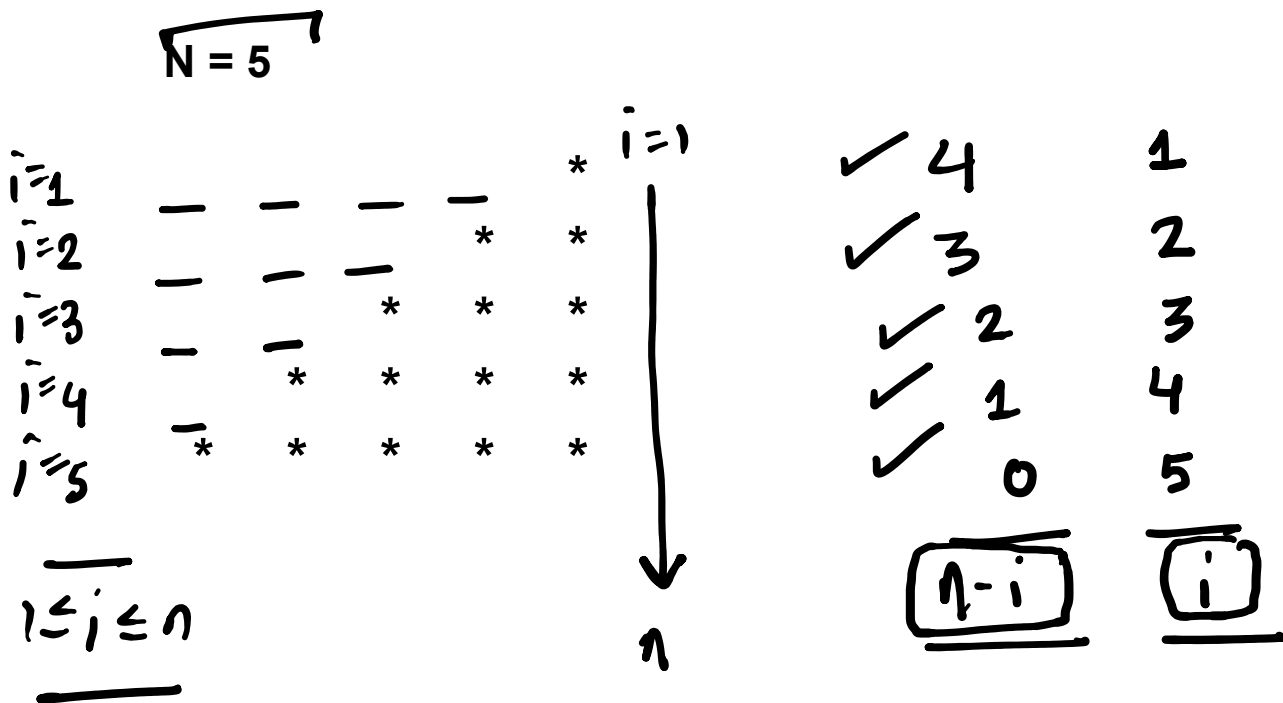
even row odd row
↓ ↓
0 1

num = ! num
or
num = 1 - num;

Pattern - 5

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example



Pattern - 6

Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

1	-	-	-	-	1
2	-	-	-	2	3
3	-	-	3	4	5
4	-	4	5	6	7
5	5	6	7	8	9

$1 \leq i \leq n$

4
3
2
1
0

$\Rightarrow \underline{n-i}$

1
2
3
4
5

$\Rightarrow \underline{i}$

↑
ing ord.
start with i

[R][HW] Pattern - 7



Given a positive number **N**, design an algorithm to **print** the pattern as shown in the example below.

Example

N = 5

```
      (← 1 →)
        2 3 2
      3 4 5 4 3
    4 5 6 7 6 5 4
  5 6 7 8 9 8 7 6 5
      .
      .
      .
      .
      1
```

Characters in C++

```
char ch;
cin >> ch;
cout << ch;
```

```
char ch;
ch = 'A';
cout << ch; // A
```

A ch
1B
=

ch
B
char ch = 'B';
cout << ch; // B

1B = 8-bits

n-bits $\Rightarrow 2^n$ nos. $\left\{ \begin{array}{l} 00 \dots 0 \Rightarrow 0 \\ \vdots \\ 11 \dots 1 \Rightarrow 2^n - 1 \end{array} \right.$

ASCII

A $\rightarrow$ 65
B $\rightarrow$ 66
:
:
Z $\rightarrow$ 90

a $\rightarrow$ 97
b $\rightarrow$ 98
c $\rightarrow$ 99
:
:
z $\rightarrow$ 122

2 2 2 2 2 2 2 2 $\Rightarrow 2^8$ nos.
0000 0000 $\rightarrow$ 0
:
1111 1111 $\rightarrow$ 255

char ch = 'd';

ascii value of 'd' = 100

0 1 1 0 0 1 0 0
7 6 5 4 3 2 1 0
8-bits

64 + 32 + 4
 $\downarrow$ $\downarrow$ $\downarrow$
2⁶ 2⁵ 2²

'0' $\rightarrow$ 48
'1' $\rightarrow$ 49
'2' $\rightarrow$ 50
'3' :
:
:
'9' $\rightarrow$ 57

ch = '8' $\rightarrow$ 8
char int

'8' - '0'
56 - 48 = 8

char ch = '6';
(digit in char form) $\rightarrow$ digit in int form

'6' $\rightarrow$ 6
'6' - '0'
54 - 48 = 6

{	$'a' \rightarrow 0$	$'a' - 'a' = 97 - 97 = 0$
	$'b' \rightarrow 1$	$'b' - 'a' = 98 - 97 = 1$
	$'c' \rightarrow 2$	$\vdots$
	$\vdots$	$\vdots$
	$\vdots$	$'z' - 'a' = 122 - 97 = 25$
	$'z' \rightarrow 25$	

i/p abcd\$ o/p: 4

i/p ab cde\$ o/p: 6

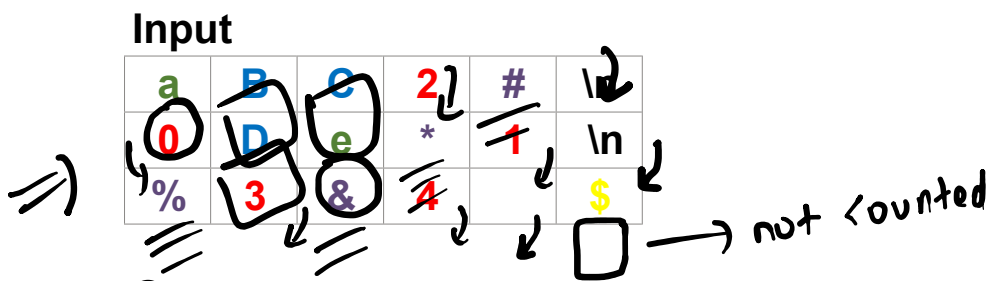
Count Types of Characters

Count Types of Characters

Given a sequence of characters, design an algorithm to **count** the number of lowercase letters, uppercase letters, digits, whitespaces and special symbols.

note : assume the character sequence ends with '\$'

Example



Output

lowercase	uppercase	digits	whitespaces	special
2	3	5	3	4

= = = = =

Minimize Path

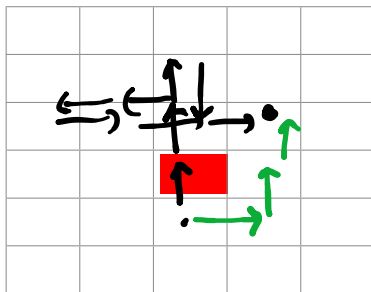
⇒

Input

N	N	N	S	W	W	E	E	E	.
---	---	---	---	---	---	---	---	---	---

Output

E	N	N
---	---	---



1,2

$\boxed{\epsilon NN}$
 or
 NNE
 or
 NEN

N
 W —+— E
 S

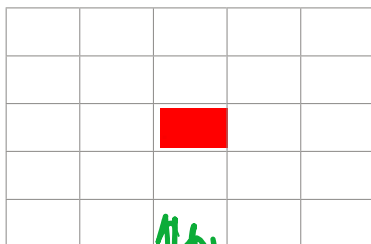
lexicographically smallest

Input

S	N	S	N	S	W	E	W
---	---	---	---	---	---	---	---

Output

S	W
---	---



SW ✓
 or
 WS

1,1

Constants in C++

→ an expr. that eval. to a fixed const
literals

int literal
str literal
float literal
char literal

```
int x=10;
cout << x; // 10
x++;
cout << x; // 11
```

```
const int y = 20;
cout << y; // 20
y++; // error
```

```
#define PI 3.14
```

const { const int z; z=100; }
const var. must be always initialized